

APP 通信协议接口设计说明书

珠海市杰理科技股份有限公司
Zhuhai Jieli Technologyco.,LTD
版权所有，未经许可，禁止外传

修改记录

版本	更新日期	描述
v1.0	2021-1-12	App 通信协议接口设计说明初始版本



目录

1. 文档介绍.....	6
1.1. 文档目的.....	6
1.2. 参考文献.....	6
1.3. 术语与缩写词.....	6
2. 功能概述.....	6
3. 其他系统/模块的调用关系.....	7
4. 文件名说明与内容安排.....	7
5. 消息模块介绍.....	7
5.1. 公共通知消息类.....	8
5.1.1. 消息介绍.....	8
5.1.2. 接口介绍.....	8
5.1.2.1. app_protocol_message_handler.....	8
5.2. OTA 通知消息类.....	9
5.2.1. 介绍.....	9
5.2.2. 接口介绍.....	9
5.2.2.1. app_protocol_ota_message_handler.....	9
5.3. GMA 私有消息.....	10
5.3.1. 消息列表.....	10
5.3.2. 接口介绍.....	10
5.3.2.1. static int gma_special_message.....	10
5.4. MMA 私有消息.....	11
5.4.1. 消息列表.....	11
这部分消息是用于处理 MMA 一些独特的命令，或者以后需要协议扩展的命令。.....	11
5.4.2. 接口介绍.....	11
5.4.2.1. mma_special_message.....	11
5.5. DMA 私有消息.....	12
5.5.1. 消息列表.....	12

5.5.2. 接口介绍.....	12
5.5.2.1. dueros_special_message.....	12
6. 与各具体协议中间层设计介绍.....	13
6.1. 概况.....	13
6.2. 参数配置接口.....	13
6.3. 协议操作函数.....	14
6.4. 协议回调接口注册函数.....	14
7. 提供给 USER 层使用的接口.....	15
7.1. 参数配置接口（在 init 之前）.....	15
7.1.1. app_protocol_set_product_id.....	15
7.1.2. app_protocol_set_vendor_id.....	15
7.1.3. app_protocol_set_local_version.....	16
7.1.4. app_protocol_set_info_group.....	16
7.1.5. app_protocol_set_tws_sibling_mac.....	16
7.2. 协议操作控制接口.....	17
7.2.1. app_protocol_ibeacon_switch.....	17
7.2.2. app_protocol_ble_adv_switch.....	17
7.2.3. app_protocol_disconnect.....	17
7.2.4. app_protocol_get_tws_data_for_lib.....	18
7.2.5. app_protocol_send_voice_data.....	18
7.2.6. app_protocol_check_connect_success.....	18
7.2.7. app_protocol_start_speech_cmd.....	19
7.2.8. app_protocol_stop_speech_cmd.....	19
7.3. 回调函数说明.....	20
7.4. APP protocol 线程接口.....	20
7.4.1. app_protocol_inti;.....	20
7.4.2. app_protocol_exit.....	21
8. USER APP 公共流程和一些接口.....	22

8.1. 公共流程.....	22
8.1.1. 唤醒语音助手流程.....	22
8.1.2. 系统事件处理流程.....	22
8.2. 接口.....	23
8.2.1. app_protocol_set_volume.....	23
8.2.2. app_protocol_get_bat_by_type.....	23
8.2.3. app_protocol_get_license_ptr.....	23
8.2.4. app_protocol_license2flash.....	24
8.2.5. app_protocol_tone_register.....	24
8.2.6. mic_rec_pram_init.....	25
8.2.7. app_protocol_handle_register.....	25
8.2.8. app_protocol_tws_sync_private_deal.....	26
8.2.9. app_protocol_tws_rx_data_private_deal.....	26
8.2.10. app_protocol_sys_event_private_deal.....	26
8.2.11. app_protocol_post_bt_event.....	27
8.2.12. app_protocol_post_app_core_callback.....	27
8.2.13. app_protocol_tws_send_to_sibling.....	27
8.2.14. app_protocol_tws_sync_send.....	28

1. 文档介绍

介绍手机 APP 通信协议的 API 接口设计，API 参数和反馈事件的意义。

1.1. 文档目的

通过本文档熟悉各个手机 APP 通信协议 API 的使用和注意事项。

1.2. 参考文献

[1].各家 AI 协议的网址和参考资料

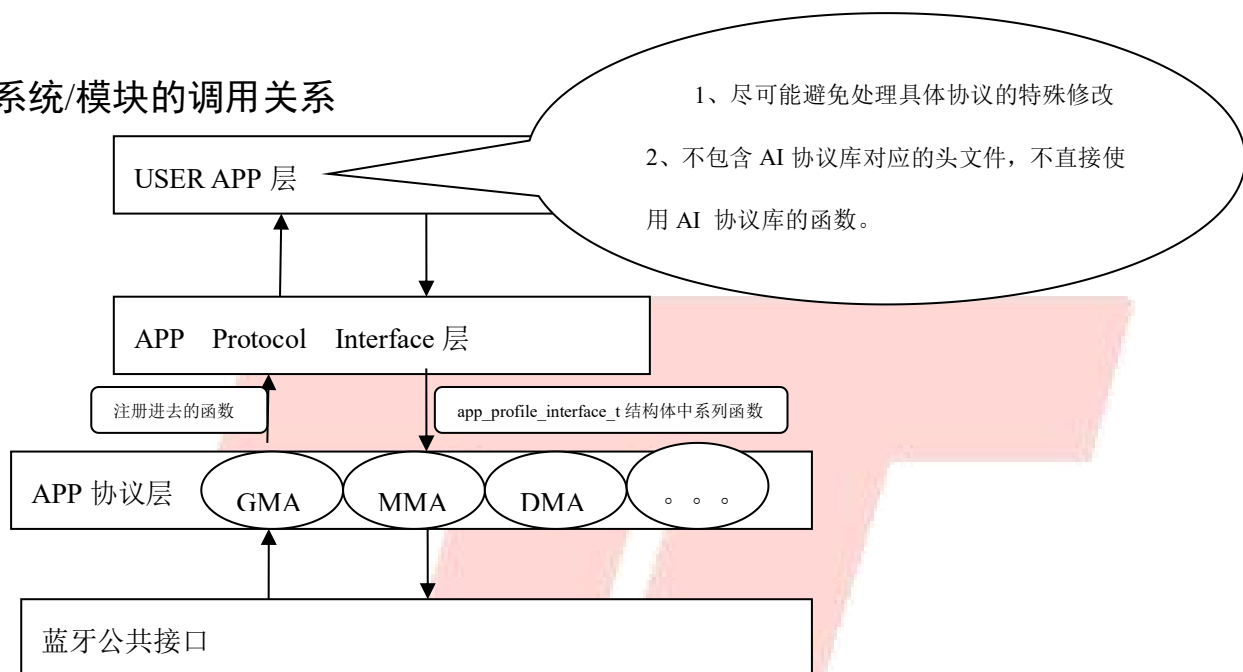
1.3. 术语与缩写词

缩写、术语	解 释
APP	手机软件
GMA	阿里天猫的 AI 协议
MMA	小米小爱的 AI 协议
DMA	百度的 AI 协议
TME	腾讯酷狗的协议

2. 功能概述

由于各个互联网公司的 AI 协议有较大差异，上层使用经常要用很多宏区分功能编译，这样就导致了一个代码的移植性很差。这个接口层设计是为上层应用提供相对比较统一的 AI APP 协议接口，屏蔽一些协议的细节,由库里面实现并抽象接口。提高上层应用代码和 AI 协议的可移植性。

3. 其他系统/模块的调用关系



4. 文件名说明与内容安排

文件名	文件内容说明
app_protocol_api.c	只能在此文件调用具体协议库的函数接口。并且抽象封装接口给 USER APP 层调用
app_protocol_common.c	USER APP 层抽象提供给 app_protocol_api 调用的接口
app_protocol_ota.c	OTA 升级的中间层接口
app_protocol_dma.c	DMA 非协议规定的通用流程和资源。如提示音，读配置文件
app_protocol_gma.c	GMA 非协议规定的通用流程和资源。如提示音，读配置文件
app_protocol_mma.c	MMA 非协议规定的通用流程和资源。如提示音，读配置文件
app_protocol_tme.c	TME 非协议规定的通用流程和资源。如提示音，读配置文件

5. 消息模块介绍

5.1. 公共通知消息类

5.1.1. 消息介绍

这类消息是每个 APP 协议都基本会有状态通知消息。

```
1. //app protocol 公共消息
2. enum {
3.     APP_PROTOCOL_COMMON_NOTICE = 0,
4.     APP_PROTOCOL_CONNECTING,    /*保留，暂未使用*/
5.     APP_PROTOCOL_CONNECTED_BLE, /*APP 通过 BLE 连接成功状态更新*/
6.     APP_PROTOCOL_CONNECTED_SPP, /*APP 通过 SPP 连接成功状态更新*/
7.     APP_PROTOCOL_DISCONNECT,    /*APP 连接断开状态更新*/
8.     APP_PROTOCOL_AUTH_PASS,     /*连接认证通过标识更新*/
9.     APP_PROTOCOL_SPEECH_START,  /*语音识别功能启动状态*/
10.    APP_PROTOCOL_SPEECH_STOP,   /*语音识别功能停止状态*/
11.    APP_PROTOCOL_SET_VOLUME,    /*app 配置音量*/
12.    APP_PROTOCOL_GET_VOLUME,    /*app 读取音量*/
13.    APP_PROTOCOL_GET_AUX_STATUS, /*保留，暂未使用*/
14.    APP_PROTOCOL_LIB_TWS_DATA_SYNC, /*需要更新给另一端 tws 数据*/
15.    APP_PROTOCOL_COMMON_NOTICE_END = 0x14F,
16. };
```

这些消息主要在 int app_protocol_message_handler(int opcode, const u8 *data, u32 len);函数中处理

5.1.2. 接口介绍

5.1.2.1. app_protocol_message_handler

函数原型	int app_protocol_message_handler(int id, int opcode, const u8 *data, u32 len);
功能描述	此函数是注册到库里面使用，处理 APP 协议的所有消息，再通过系统消息接口的方式发给其它线程处理
参数说明	*\param[in] id //连上的协议 ID 值 *\param[in] opcode //app_profile_api.h 里面的枚举值 *\param[in] data //可能状态会有其它的参数，没有是 NULL

	*\param[in] len //状态其它的参数的长度，没有是 0
输出	返回值是错误码
例子	
补充说明	

5.2. OTA 通知消息类

5.2.1. 介绍

这类消息是每个 APP 协议进行 OTA 升级时抽象出来的消息，用于通知上层状态和进度

```
1. enum {
2.     APP_PROTOCOL_OTA_COMMON_NOTICE = APP_PROTOCOL_COMMON_NOTICE_END + 1,
3.     APP_PROTOCOL_OTA_CHECK,
4.     APP_PROTOCOL_OTA_GET_APP_VERSION,
5.     APP_PROTOCOL_OTA_CHECK_CRC,
6.     APP_PROTOCOL_OTA_BEGIN,
7.     APP_PROTOCOL_OTA_TRANS_DATA,
8.     APP_PROTOCOL_OTA_PERCENT,
9.     APP_PROTOCOL_OTA_END,
10.    APP_PROTOCOL_OTA_SUCCESS,
11.    APP_PROTOCOL_OTA_FAIL,
12.    APP_PROTOCOL_OTA_CANCEL,
13.    APP_PROTOCOL_OTA_REBOOT,
14.    APP_PROTOCOL_OTA_COMMON_NOTICE_END = 0xFF,
15.};
```

这些消息主要在 int app_ota_message_handler(int opcode, const u8 *data, u32 len)函数中处理

5.2.2. 接口介绍

5.2.2.1. app_protocol_ota_message_handler

函数原型	int app_protocol_ota_message_handler(int id, int opcode, u8 *data, u32 len)
------	---

版权所有，侵权必究

9

功能描述	此函数在 app_protocol_message_handler 中调用，判断范围后，把对应的消息给到 app_protocol_ota_message_handler 处理，
参数说明	*\param[in] id //区分什么协议的 ID 值 *\param[in] opcode //app_profile_api.h 里面关于 OTA 的枚举值 *\param[in] data //可能状态会有其它的参数，没有是 NULL *\param[in] len //状态其它的参数的长度，没有是 0
输出	返回值是错误码
例子	
补充说明	

5.3.GMA 私有消息

5.3.1. 消息列表

这部分消息是用于处理 GMA 一些独特的命令，或者以后需要协议扩展的命令。

```
1. //GMA 私有消息
2. enum {
3.     APP_PROTOCOL_GMA_NOTICE_BEGIN          = GMA_HANDLER_ID,
4.     APP_PROTOCOL_GMA_FMTX_SETFRE,
5.     APP_PROTOCOL_GMA_FMTX_GETFRE,
6.     APP_PROTOCOL_GMA_NOTICE_END            = GMA_HANDLER_ID + 0xFF,
7. };
```

这些消息主要在 int gma_message_handler(int opcode, const u8 *data, u32 len)函数中处理

5.3.2. 接口介绍

5.3.2.1. static int gma_special_message

函数原型	static int gma_special_message(int id, int opcode, u8 *data, u32 len)
功能描述	定义一个协议特殊的消息函数是为了减少代码量，不会编译到不需要协议库
参数说明	*\param[in] id //区分什么协议的 ID 值

	*\param[in] opcode //app_profile_api.h 里面关于 OTA 的枚举值 *\param[in] data //可能状态会有其它的参数，没有是 NULL *\param[in] len //状态其它的参数的长度，没有是 0
输出	返回值是错误码
例子	
补充说明	

5.4.MMA 私有消息

5.4.1. 消息列表

这部分消息是用于处理 MMA 一些独特的命令，或者以后需要协议扩展的命令。

```

1. //MMA 私有消息
2. enum {
3.     APP_PROTOCOL_MMA_NOTICE           = MMA_HANDLER_ID,
4.     APP_PROTOCOL_MMA_SAVE_INFO,
5.     APP_PROTOCOL_MMA_READ_INFO,
6.     APP_PROTOCOL_MMA_SAVE_ADV_COUNTER,
7.     APP_PROTOCOL_MMA_READ_ADV_COUNTER,
8.     APP_PROTOCOL_MMA_NOTICE_END       = MMA_HANDLER_ID + 0xFF,
9. };

```

5.4.2. 接口介绍

5.4.2.1. mma_special_message

函数原型	static int mma_special_message(int id, int opcode, u8 *data, u32 len)
功能描述	定义一个协议特殊的消息函数是为了减少代码量，不会编译到不需要协议库
参数说明	*\param[in] id //区分什么协议的 ID 值 *\param[in] opcode //app_profile_api.h 里面关于 OTA 的枚举值 *\param[in] data //可能状态会有其它的参数，没有是 NULL

	*\param[in] len //状态其它的参数的长度，没有是 0
输出	返回值是错误码
例子	
补充说明	

5.5.DMA 私有消息

5.5.1. 消息列表

这部分消息是用于处理 MMA 一些独特的命令，或者以后需要协议扩展的命令

```

1. //DMA 私有消息
2. enum {
3.     APP_PROTOCOL_DMA_NOTICE           = DMA_HANDLER_ID,
4.     APP_PROTOCOL_DMA_SAVE_RAND,
5.     APP_PROTOCOL_DMA_READ_RAND,
6.     APP_PROTOCOL_DMA_TWS_SNED_RAND,
7.     APP_PROTOCOL_DMA_NOTICE_END      = DMA_HANDLER_ID + 0xFF,
8. };

```

5.5.2. 接口介绍

5.5.2.1. dueros_special_message

函数原型	static int dueros_special_message(int id, int opcode, u8 *data, u32 len)
功能描述	定义一个协议特殊的消息函数是为了减少代码量，不会编译到不需要协议库
参数说明	*\param[in] id //区分什么协议的 ID 值 *\param[in] opcode //app_profile_api.h 里面关于 OTA 的枚举值 *\param[in] data //可能状态会有其它的参数，没有是 NULL *\param[in] len //状态其它的参数的长度，没有是 0
输出	返回值是错误码
例子	

6. 与各具体协议中间层设计介绍

6.1. 概况

每个协议的实现提供的接口需要根据 `app_profile_interface_t` 结构体来完成。为了能够支持同时支持多个协议组合，比如 (DMA+JL rcsp ota)，会有一个链表把这些结构体组织起来。每个协议会固定一个 UUID，用于查找对应的协议接口组。但是协议只能支持一个连接。

```
1. #define DEMO_HANDLER_ID    0x300    /*作为一个使用的例子,同时也可作为客户自己添加协议的 ID*/
2. #define GMA_HANDLER_ID    0x400    /*阿里天猫协议接口 ID*/
3. #define MMA_HANDLER_ID    0x500    /*小米 MMA 协议接口 ID*/
4. #define DMA_HANDLER_ID    0x600    /*百度 DMA 协议接口 ID*/
5. #define TME_HANDLER_ID    0x700    /*腾讯酷狗 TME 协议接口 ID*/
6. #define AMA_HANDLER_ID    0x800    /*亚马逊的 AMA 协议接口 ID*/
```

`list_for_each_app_profile` 可以遍历所有支持的协议

6.2. 参数配置接口

```
1. struct app_protocol_info_t {
2.     /*配置类接口*/
3.     void (*set_product_id)(u32 pid);
4.     void (*set_vendor_id)(u32 vid);
5.     void (*set_local_version)(u32 vid);
6.     void (*set_special_info_group)(void *addr);
7.     void (*set_tws_sibling_mac)(void *mac);
8. };
```

6.3. 协议操作函数

```
1. struct app_ctrl_operation_t {  
2.     int(*protocol_init)();  
3.     int(*protocol_exit)();  
4.     int(*adv_enable)(int enable);  
5.     int(*ibeacon_adv)(int sw);  
6.     int(*regist_wakeup_send)(void *priv, void *cbk);  
7.     int(*regist_recieve_cbk)(void *priv, void *cbk);  
8.     int(*latency_enable)(void *priv, u32 enable);  
9.     int(*send_data)(void *priv, void *buf, u16 len);  
10.    int(*send_voice_data)(void *buf, u16 len);  
11.    int(*disconnect)(void *addr);  
12.    int(*twis_receive_sync_data)(u8 *data, int len);  
13.    int(*get_auth_state)(void);  
14.    int(*start_voice_recognition)(int st);  
15. };
```

一般考虑连接的时候只有一个 APP 处于连接状态，所以发送的时候只有一个协议可以发送成功。

```
void app_protocol_send_data(void *buf, u16 len)  
{  
    const app_profile_interface_t *app;  
    list_for_each_app_profile(app){  
        if(app->app_conn.send_data){  
            app->app_conn.send_data(NULL, buf, len);  
        }  
    }  
}
```

6.4. 协议回调接口注册函数

```
1. struct callback_register_t {  
2.     void(*message_handler_regedit)(int(*handler)(int id, int opcode, u8 *data, u32 len));  
3.     void(*check_twis_role_is_master_register)(bool (*handler)(void));  
4.     void(*tx_resume)(void (*handler)(void));  
5.     void(*rx_resume)(void (*handler)(void));  
6.     void(*get_battery)(bool (*handler)(u8 battery_type, u8 *value));  
7. };
```

蓝牙部门负责的 APP 协议开发要实现这三个结构体。提供对应参数的函数赋值给结构体使用。

7. 提供给 USER 层使用的接口

7.1. 参数配置接口（在 init 之前）

7.1.1. app_protocol_set_product_id

函数原型	void app_protocol_set_product_id(u32 handler_id, u32 pid);
功能描述	设置产品的 product ID，需要跟相应协议的厂商提出申请
参数说明	*\param[in] handler_id //每个协议指定的接口标识 *\param[in] pid //产品的 ID，需要申请的值
输出	
例子	
关联模块	
补充说明	

7.1.2. app_protocol_set_vendor_id

函数原型	void app_protocol_set_vendor_id(u32 handler_id, u32 vid)
功能描述	设置产品的 vendor ID，需要跟相应协议的厂商提出申请
参数说明	*\param[in] handler_id //每个协议指定的接口标识 *\param[in] vid //厂商的 ID，需要申请的值
输出	
例子	
关联模块	
补充说明	

7.1.3. app_protocol_set_local_version

函数原型	void app_protocol_set_local_version(u32 handler_id, u32 version)
功能描述	设置产品的当前软件版本，用于 app 显示和升级检查
参数说明	*\param[in] handler_id //每个协议指定的接口标识 *\param[in] version //自定义值，格式库内会自己根据协议进行调整
输出	
例子	
补充说明	

7.1.4. app_protocol_set_info_group

函数原型	void app_protocol_set_info_group(u32 handler_id, void *addr)
功能描述	配置协议更多信息，例如 GMA 的三元组地址，百度的三月组。
参数说明	*\param[in] handler_id //每个协议指定的接口标识 *\param[in] addr //信息的其实地址
输出	
例子	
补充说明	这个接口涉及到一个数据格式，可能每个协议有差异。协议开发定义好数据结构，让 app 层根据结构补充信息。不能随意更改数据结构

7.1.5. app_protocol_set_tws_sibling_mac

函数原型	void app_protocol_set_tws_sibling_mac(u8 *mac)
功能描述	配置 TWS 的地址,app 协议用于配置广播包或者命令告知 app
参数说明	*\param[in] mac //TWS 另一端的地址
输出	
例子	

补充说明

7.2. 协议操作控制接口

7.2.1. app_protocol_ibeacon_switch

函数原型	void app_protocol_ibeacon_switch(int sw)
功能描述	有些协议需要开 beacon 包的
参数说明	*\param[in] sw //开关标识 1 或者 0
输出	
例子	
补充说明	

7.2.2. app_protocol_ble_adv_switch

函数原型	void app_protocol_ble_adv_switch(int sw)
功能描述	协议开关广播包操作接口
参数说明	*\param[in] sw //开关标识 1 或者 0
输出	
例子	
补充说明	

7.2.3. app_protocol_disconnect

函数原型	void app_protocol_disconnect(void *addr)
功能描述	断开 app 的连接
参数说明	*\param[in] add //目前单台应用，可传 NULL

输出	
例子	
补充说明	

7.2.4. app_protocol_get_tws_data_for_lib

函数原型	void app_protocol_get_tws_data_for_lib(u8 *data, u32 len)
功能描述	Tws 收到另一端的数据调用这个接口把数据完整传回去库里面解析
参数说明	*\param[in] data //收到数据的地址 *\param[in] len //收到数据的长度
输出	
例子	
补充说明	

7.2.5. app_protocol_send_voice_data

函数原型	int app_protocol_send_voice_data(uint8_t *voice_buf, uint16_t voice_len)
功能描述	发送语音数据给 APP,
参数说明	*\param[in] voice_buf //语音数据的起始地址 *\param[in] voice_len //语音数据的长度
输出	
例子	
补充说明	

7.2.6. app_protocol_check_connect_success

函数原型	int app_protocol_check_connect_success()
功能描述	查询当前的协议有没有 APP 连接成功

参数说明	
输出	返回值是当前的连接状态
例子	
补充说明	

7.2.7. app_protocol_start_speech_cmd

函数原型	int app_protocol_start_speech_cmd()
功能描述	发命令启动语音识别功能
参数说明	
输出	返回值是当前的命令有没有发送成功
例子	
补充说明	

7.2.8. app_protocol_stop_speech_cmd

函数原型	int app_protocol_stop_speech_cmd()
功能描述	发命令停止语音识别功能
参数说明	
输出	返回值是当前的命令有没有发送成功
例子	
补充说明	

7.3. 回调函数说明

函数名称	简要说明
message_handler_regedit	公共消息函数传递到库里面调用
check_tws_role_is_master_register	库需要检查 TWS 的主从状态
tx_resume	可能需要唤醒线程处理去发数
rx_resume	可能需要唤醒线程去处理收到的数据
get_battery	获取各种电池电量的总接口

7.4. APP protocol 线程接口

7.4.1. app_protocol_inti;

函数原型	void app_protocol_inti(int handler_id);
功能描述	/*这个接口主要是建立一个线程,注册一些协议需要用的公共接口,比如 resume, message_handler 等函数。建立了线程之后, 初始化对应的协议*/
参数说明	*\param[in] handler_id //头文件指定协议的一个 ID 值
输出	
例子	在对应的路径加上头文件#include "app_protocol_api.h" app_protocol_init(DEMO_HANDLER_ID);
补充说明	<pre> #define GMA_HANDLER_ID 0x400 /*阿里天猫协议接口 ID*/ #define MMA_HANDLER_ID 0x500 /*小米 MMA 协议接口 ID*/ #define DMA_HANDLER_ID 0x600 /*百度 DMA 协议接口 ID*/ #define TME_HANDLER_ID 0x700 /*腾讯酷狗 TME 协议接口 ID*/ </pre>

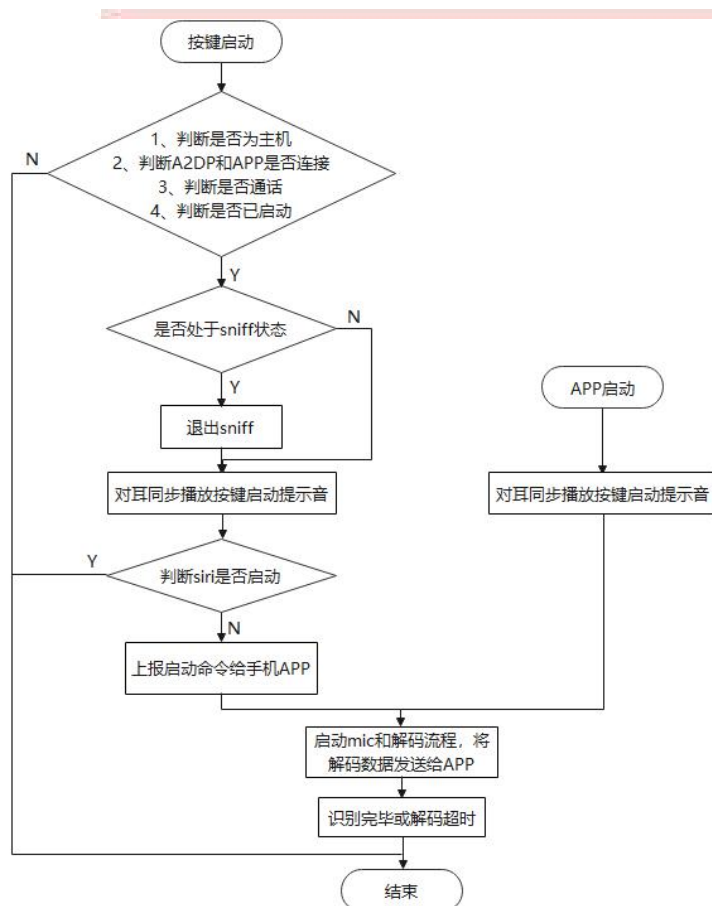
7.4.2. app_protocol_exit

函数原型	void app_protocol_inti(int handler_id);
功能描述	统一的释放 AI 协议接口，主要是删除线程操作，在删除完之后再释放对应 APP 协议栈的资源
参数说明	*\param[in] handler_id //头文件指定协议的一个 ID 值
输出	
例子	
补充说明	

8. USER APP 公共流程和一些接口

8.1. 公共流程

8.1.1. 唤醒语音助手流程



8.1.2. 系统事件处理流程

第三方协议系统事件的统一在 `app_protocol_sys_event_handler` 函数里处理。公共的处理流程在 `app_protocol_common.c` 里面执行，非公共部分的处理流程，在各自的 `app_protocol_xxx.c` 文件里处理。

8.2. 接口

8.2.1. app_protocol_set_volume

函数原型	int app_protocol_set_volume(u8 vol);
功能描述	设置设备的音量级别，
参数说明	*\param[in] vol //目标音量
输出	
例子	
补充说明	

8.2.2. app_protocal_get_bat_by_type

函数原型	u8 app_protocal_get_bat_by_type(u8 type)
功能描述	提供给库调用的获取各种电量信息的接口
参数说明	*\param[in] type //要获取的电量类型
输出	电量大小或充电标志，范围为 0-100。获取不到时返回 0
例子	
补充说明	

8.2.3. app_protocal_get_license_ptr

函数原型	const u8 *app_protocal_get_license_ptr(void);
功能描述	获取 flash 里三元组信息的头指针
参数说明	
输出	三元组信息的头指针
例子	

补充说明

8.2.4. app_protocol_license2flash

函数原型	int app_protocol_license2flash(const u8 *data, u16 len);
功能描述	将三元组数据写到 flash 的末尾处
参数说明	*\param[in] data //要保存的数据 *\param[in] data //要保存的数据长度
输出	写入成功返回 0，失败会返回-1
例子	
补充说明	

8.2.5. app_protocol_tone_register

函数原型	void app_protocol_tone_register(const char **tone_table);
功能描述	注册或切换第三方协议提示音表
参数说明	*\param[in] tone_table //提示音表
输出	
例子	
补充说明	目前加了 8 个可能用到的提示音： APP_PROTOCOL_TONE_CONNECTED_ALL_FINISH, //经典蓝牙和协议已连接 APP_PROTOCOL_TONE_PROTOCOL_CONNECTED, //协议已连接 APP_PROTOCOL_TONE_CONNECTED_NEED_OPEN_APP, //经典蓝牙已连接，但协议未连接 APP_PROTOCOL_TONE_DISCONNECTED, //经典蓝牙已断开 APP_PROTOCOL_TONE_DISCONNECTED_ALL, //经典蓝牙和协议未连接 APP_PROTOCOL_TONE_OPEN_APP, //按键启动语音助手时，APP 未连接 APP_PROTOCOL_TONE_SPEECH_APP_START, //APP 启动语音助手提示音

	APP_RROTOCOL_TONE_SPEECH_KEY_START, // 按键启动语音助手提示音 APP_RROTOCOL_TONE_MAX,
--	--

8.2.6. mic_rec_pram_init

函数原型	int mic_rec_pram_init(u32 enc_type, u8 opus_type, u16(*speech_send)(u8 *buf, u16 len), u16 frame_num, u16 cbuf_size);
功能描述	初始化语音编码的参数
参数说明	*\param[in] enc_type // 编码类型，opus 或者 speex *\param[in] opus_type // opus 的类型，标准或者 TME 特殊格式 *\param[in] speech_send // 编码后发送到 APP 的接口 *\param[in] frame_num // 每次发送的解码帧数 *\param[in] cbuf_size // 解码的缓存 buf
输出	初始化结果。成功返回 0，失败返回非 0
例子	
补充说明	

8.2.7. app_protocol_handle_register

函数原型	void app_protocol_handle_register(struct app_protocol_private_handle_t *hd);
功能描述	第三方协议非公共事件、消息处理函数注册接口
参数说明	*\param[in] hd // 特殊处理函数句柄
输出	

例子	
补充说明	

8.2.8. app_protocol_tws_sync_private_deal

函数原型	void app_protocol_tws_sync_private_deal(int cmd, int value);
功能描述	非公共对耳同步事件的处理函数
参数说明	*\param[in] cmd //对耳同步执行的命令 *\param[in] value //对耳同步执行的命令参数
输出	
例子	
补充说明	

8.2.9. app_protocol_tws_rx_data_private_deal

函数原型	void app_protocol_tws_rx_data_private_deal(u16 opcode, u8 *data, u16 len);
功能描述	非公共的对耳数据处理函数
参数说明	*\param[in] opcode //收到来自对耳的命令 *\param[in] data //收到来自对耳的数据 *\param[in] len //数据长度
输出	
例子	
补充说明	

8.2.10. app_protocol_sys_event_private_deal

函数原型	int app_protocol_sys_event_private_deal(struct sys_event *event);
功能描述	非公共的系统事件处理函数
参数说明	*\param[in] event //系统事件

输出	
例子	
补充说明	

8.2.11. app_protocol_post_bt_event

函数原型	void app_protocol_post_bt_event(u8 event, void *priv);
功能描述	发送第三方协议蓝牙事件
参数说明	*\param[in] event //蓝牙事件 *\param[in] event //蓝牙事件的参数
输出	
例子	
补充说明	

8.2.12. app_protocol_post_app_core_callback

函数原型	int app_protocol_post_app_core_callback(int callback, void *priv);
功能描述	发送回调函数到 app_core 任务中处理
参数说明	*\param[in] callback //回调函数 *\param[in] priv //回调函数的参数
输出	返回 0 表示发送成功，非 0 表示失败
例子	
补充说明	

8.2.13. app_protocol_tws_send_to_sibling

函数原型	int app_protocol_tws_send_to_sibling(u16 opcode, u8 *data, u16 len);
功能描述	发送命令和数据给对耳
参数说明	*\param[in] opcode //收到来自对耳的命令

	<code>*\param[in] data</code> //收到来自对耳的数据 <code>*\param[in] len</code> //数据长度
输出	返回 0 表示发送成功，非 0 表示失败
例子	
补充说明	

8.2.14. app_protocol_tws_sync_send

函数原型	<code>int app_protocol_tws_sync_send(int cmd, int value);</code>
功能描述	发送对耳同步执行的命令
参数说明	<code>*\param[in] cmd</code> //对耳同步执行的命令 <code>*\param[in] value</code> //对耳同步执行的命令参数
输出	返回 0 表示发送成功，非 0 表示失败
例子	
补充说明	